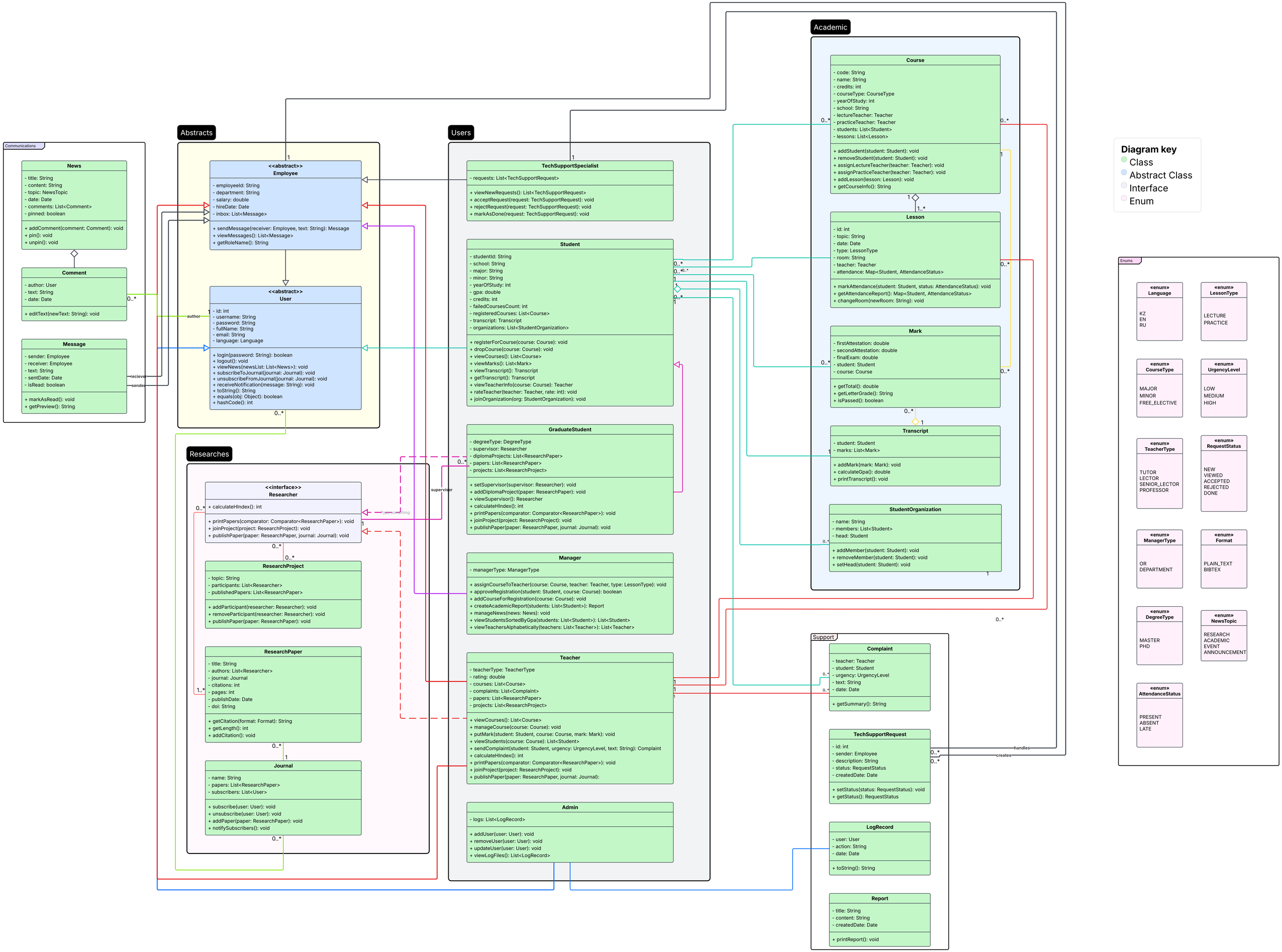
Communications
News
- title: String
- content: String
- topic: NewsTopic
- date: Date
- comments: List<Comment>
- pinned: boolean
+ addComment(comment: Comment): void
+ pin(): void
+ unpin(): void
Comment
- author: User
- text: String
- date: Date
+ editText(newText: String): void
Message
- sender: Employee
- receiver: Employee
- text: String
- sentDate: Date
- isRead: boolean
+ markAsRead(): void
+ getPreview(): String
Abstracts
<<abstract>>
Employee
- employeeId: String
- department: String
- salary: double
- hireDate: Date
- inbox: List<Message>
+ sendMessage(receiver: Employee, text: String): Message
+ viewMessages(): List<Message>
+ getRoleName(): String
<<abstract>>
User
- id: int
- username: String
- password: String
- fullName: String
- email: String
- language: Language
+ login(password: String): boolean
+ logout(): void
+ viewNews(newsList: List<News>): void
+ subscribeToJournal(journal: Journal): void
+ unsubscribeFromJournal(journal: Journal): void
+ receiveNotification(message: String): void
+ toString(): String
+ equals(obj: Object): boolean
+ hashCode(): int
author
reciever
sender
Researches
<<interface>>
Researcher
+ calculateHIndex(): int
+ printPapers(comparator: Comparator<ResearchPaper>): void
+ joinProject(project: ResearchProject): void
+ publishPaper(paper: ResearchPaper, journal: Journal): void
ResearchProject
- topic: String
- participants: List<Researcher>
- publishedPapers: List<ResearchPaper>
+ addParticipant(researcher: Researcher): void
+ removeParticipant(researcher: Researcher): void
+ publishPaper(paper: ResearchPaper): void
ResearchPaper
- title: String
- authors: List<Researcher>
- journal: Journal
- citations: int
- pages: int
- publishDate: Date
- doi: String
+ getCitation(format: Format): String
+ getLength(): int
+ addCitation(): void
Journal
- name: String
- papers: List<ResearchPaper>
- subscribers: List<User>
+ subscribe(user: User): void
+ unsubscribe(user: User): void
+ addPaper(paper: ResearchPaper): void
+ notifySubscribers(): void
supervisor
Users
TechSupportSpecialist
- requests: List<TechSupportRequest>
+ viewNewRequests(): List<TechSupportRequest>
+ acceptRequest(request: TechSupportRequest): void
+ rejectRequest(request: TechSupportRequest): void
+ markAsDone(request: TechSupportRequest): void
Student
- studentId: String
- school: String
- major: String
- minor: String
- yearOfStudy: int
- gpa: double
- credits: int
- failedCoursesCount: int
- registeredCourses: List<Course>
- transcript: Transcript
- organizations: List<StudentOrganization>
+ registerForCourse(course: Course): void
+ dropCourse(course: Course): void
+ viewCourses(): List<Course>
+ viewMarks(): List<Mark>
+ viewTranscript(): Transcript
+ getTranscript(): Transcript
+ viewTeacherInfo(course: Course): Teacher
+ rateTeacher(teacher: Teacher, rate: int): void
+ joinOrganization(org: StudentOrganization): void
GraduateStudent
- degreeType: DegreeType
- supervisor: Researcher
- diplomaProjects: List<ResearchPaper>
- papers: List<ResearchPaper>
- projects: List<ResearchProject>
+ setSupervisor(supervisor: Researcher): void
+ addDiplomaProject(paper: ResearchPaper): void
+ viewSupervisor(): Researcher
+ calculateHIndex(): int
+ printPapers(comparator: Comparator<ResearchPaper>): void
+ joinProject(project: ResearchProject): void
+ publishPaper(paper: ResearchPaper, journal: Journal): void
Manager
- managerType: ManagerType
+ assignCourseToTeacher(course: Course, teacher: Teacher, type: LessonType): void
+ approveRegistration(student: Student, course: Course): boolean
+ addCourseForRegistration(course: Course): void
+ createAcademicReport(students: List<Student>): Report
+ manageNews(news: News): void
+ viewStudentsSortedByGpa(students: List<Student>): List<Student>
+ viewTeachersAlphabetically(teachers: List<Teacher>): List<Teacher>
Teacher
- teacherType: TeacherType
- rating: double
- courses: List<Course>
- complaints: List<Complaint>
- papers: List<ResearchPaper>
- projects: List<ResearchProject>
+ viewCourses(): List<Course>
+ manageCourse(course: Course): void
+ putMark(student: Student, course: Course, mark: Mark): void
+ viewStudents(course: Course): List<Student>
+ sendComplaint(student: Student, urgency: UrgencyLevel, text: String): Complaint
+ calculateHIndex(): int
+ printPapers(comparator: Comparator<ResearchPaper>): void
+ joinProject(project: ResearchProject): void
+ publishPaper(paper: ResearchPaper, journal: Journal):
Admin
- logs: List<LogRecord>
+ addUser(user: User): void
+ removeUser(user: User): void
+ updateUser(user: User): void
+ viewLogFiles(): List<LogRecord>
Academic
Course
- code: String
- name: String
- credits: int
- courseType: CourseType
- yearOfStudy: int
- school: String
- lectureTeacher: Teacher
- practiceTeacher: Teacher
- students: List<Student>
- lessons: List<Lesson>
+ addStudent(student: Student): void
+ removeStudent(student: Student): void
+ assignLectureTeacher(teacher: Teacher): void
+ assignPracticeTeacher(teacher: Teacher): void
+ addLesson(lesson: Lesson): void
+ getCourseInfo(): String
Lesson
- id: int
- topic: String
- date: Date
- type: LessonType
- room: String
- teacher: Teacher
- attendance: Map<Student, AttendanceStatus>
+ markAttendance(student: Student, status: AttendanceStatus): void
+ getAttendanceReport(): Map<Student, AttendanceStatus>
+ changeRoom(newRoom: String): void
Mark
- firstAttestation: double
- secondAttestation: double
- finalExam: double
- student: Student
- course: Course
+ getTotal(): double
+ getLetterGrade(): String
+ isPassed(): boolean
Transcript
- student: Student
- marks: List<Mark>
+ addMark(mark: Mark): void
+ calculateGpa(): double
+ printTranscript(): void
StudentOrganization
- name: String
- members: List<Student>
- head: Student
+ addMember(student: Student): void
+ removeMember(student: Student): void
+ setHead(student: Student): void
Support
Complaint
- teacher: Teacher
- student: Student
- urgency: UrgencyLevel
- text: String
- date: Date
+ getSummary(): String
TechSupportRequest
- id: int
- sender: Employee
- description: String
- status: RequestStatus
- createdDate: Date
+ setStatus(status: RequestStatus): void
+ getStatus(): RequestStatus
handles
creates
LogRecord
- user: User
- action: String
- date: Date
+ toString(): String
Report
- title: String
- content: String
- createdDate: Date
+ printReport(): void
Diagram key
Class
Abstract Class
Interface
Enum
Enums
«enum» Language
KZ
EN
RU
«enum» LessonType
LECTURE
PRACTICE
«enum» CourseType
MAJOR
MINOR
FREE_ELECTIVE
«enum» UrgencyLevel
LOW
MEDIUM
HIGH
«enum» TeacherType
TUTOR
LECTOR
SENIOR_LECTOR
PROFESSOR
«enum» RequestStatus
NEW
VIEWED
ACCEPTED
REJECTED
DONE
«enum» ManagerType
OR
DEPARTMENT
«enum» Format
PLAIN_TEXT
BIBTEX
«enum» DegreeType
MASTER
PHD
«enum» NewsTopic
RESEARCH
ACADEMIC
EVENT
ANNOUNCEMENT
«enum» AttendanceStatus
PRESENT
ABSENT
LATE